

CODE ABAP2

1주차 멘토링

25.12.17 ~ 25.12.18

CL3 이창근

SAP

TRENDS

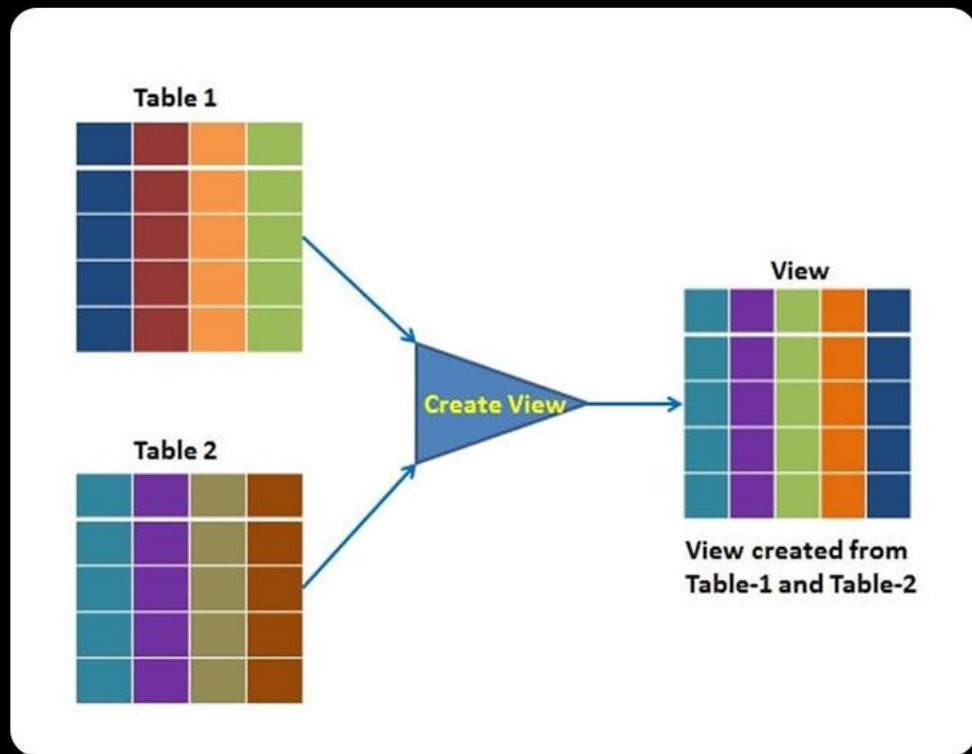
AUDIT

ESMENT

목차

- ABAP 딕셔너리
- ABAP 프로그램
- AT SELECTION-SCREEN OUTPUT (실습)
- 매개변수가 있는 서브루틴 (실습)
- SIGN E OPTION EQ ≠ SIGN I OPTION NE인 이유

Table 과 View 의 궁극적 차이



- **Table** 은 데이터가 실제로 저장되는 공간이며, 데이터베이스에 영구적으로 공간을 차지한다.
- **View** 는 필요한 데이터만 함께 모아 보기 위한 것일 뿐이며, 실제 공간을 차지하지는 않는다.
- **Maintenance View** 는 유지보수를 위한 뷰로, 데이터를 생성, 수정, 삭제할 수 있는 역할을 수행

Table 의 기본 구조

자유게시판 →
(글 n개)



게시물 번호

제목

내용

게시일자

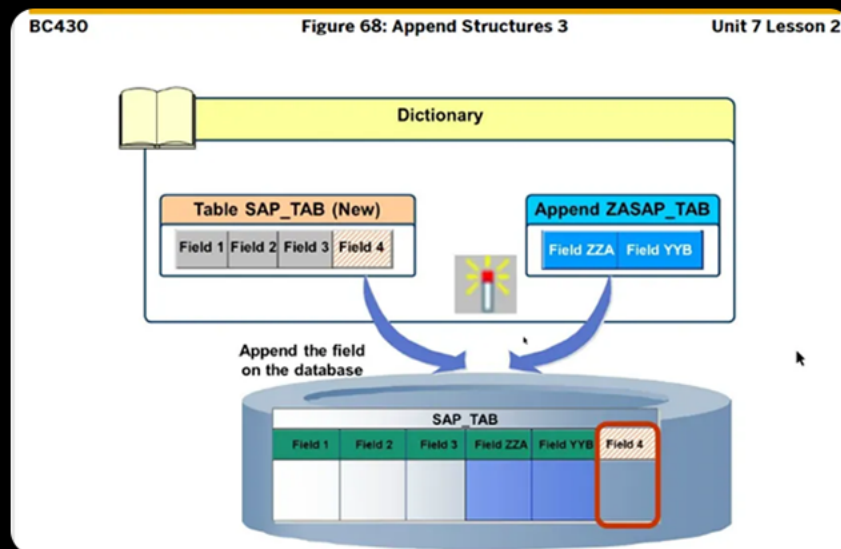
조회수 등

Table 의 기본 구조

- 어떤 객체의 각 구성요소를 필드(컬럼)에 매칭시키면, 그 객체는 곧 여러 개의 필드를 가진 테이블과 동일하다
 - 각 필드에는 데이터 타입과 데이터 길이, Description을 가져야 한다.
 - ★ 이 3가지 속성은 여러가지 방법으로 정의할 수 있다.
 1. Table 에 직접 Built-in으로 정의 → 실무에서 가장 안 쓰는 방법
 2. Data Element 를 사용하여 정의
 3. Data Element 와 Domain 을 같이 사용하여 정의

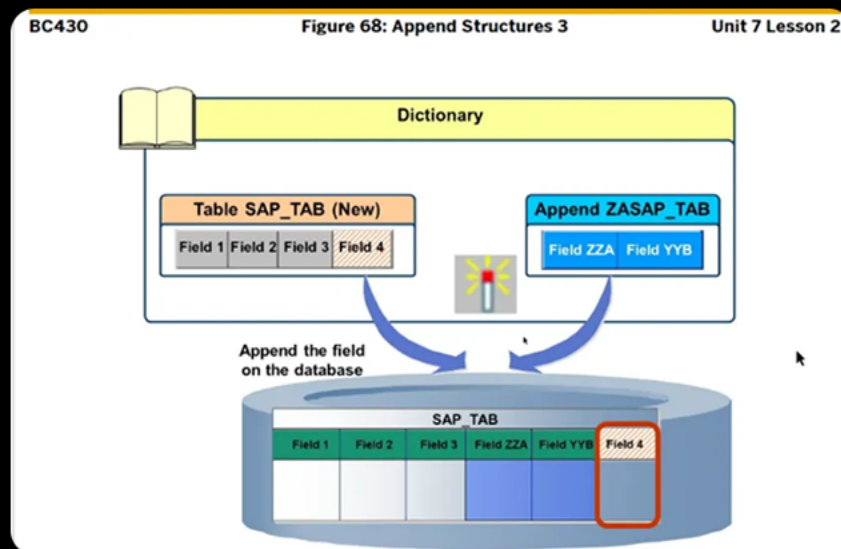
Structure

- **Structure** 는 1차원 구조이다.
- 데이터의 구조만 가지고 있을 뿐, **실제 데이터를 저장하지 않는다**.
- **.include** 나 **.append** 를 사용하여 테이블에 특정 데이터 구조를 추가할 수 있다.
 - **.include** 는 어떤 테이블에 붙였다하더라도 해당 테이블에 종속되지 않으며, 원하는 위치에 어디든 붙일 수 있다.
 - **.append** 는 **.include** 와 다르게, 생성과 동시에 해당 테이블에 종속되며, 테이블의 맨 마지막에만 붙일 수 있다.



Structure

- **Structure** 는 1차원 구조이다.
- 데이터의 구조만 가지고 있을 뿐, **실제 데이터를 저장하지 않는다**.
- **.include** 나 **.append** 를 사용하여 테이블에 특정 데이터 구조를 추가할 수 있다.
 - **.include** 는 어떤 테이블에 붙였다하더라도 해당 테이블에 종속되지 않으며, 원하는 위치에 어디든 붙일 수 있다.
 - **.append** 는 **.include** 와 다르게, 생성과 동시에 해당 테이블에 종속되며, 테이블의 맨 마지막에만 붙일 수 있다.



View Cluster

- 복잡한 데이터 구조를 유지보수하기 위한 특수한 객체이다.
 - 여러 개의 연관된 테이블 혹은 View들을 하나의 논리적 단위로 묶어서 관리
 - 회사 코드, 플랜트, 저장 위치 등 계층적 조직 데이터 관리
 - 복잡한 설정 값들의 그룹을 유지보수
- 각 View에 필드 종속을 설정하여 마치 윈도우의 파일 탐색기처럼 계층형 구조를 만들어 뷰를 모아둘 수 있다.
 - 이번 과정에서는 여러 개의 `Maintenance View` 를 View Cluster에 구성하여 사용했다.

Variable (변수)

- 데이터(값)을 저장하기 위한 공간.
 - 변수를 선언할 때는 **데이터 타입, 데이터 길이**를 정의해야 한다.
 - 변수명에는 `gv_` `lv_` `pv_` 등의 **접두사(Prefix)**를 꼭 써주어야 한다.
 - 변수명 또한 **적당한 길이**로 작성하며, 누가 보아도 어떤 역할을 하는지 알아볼 수 있게 작성한다.

ex) `gv_cnt` `gv_userName` `pv_chkNum`

```
DATA gv_cnt TYPE i.  
DATA gv_var1 TYPE zvc13_20_kind. "Data Element를 Type으로 사용  
DATA gv_var2 TYPE scarr-carrid. "Structure Type(Table Field)를 Type으로 사용  
DATA: gv_var4 TYPE i,  
      gv_var5 TYPE i,  
      gv_var6 TYPE i. " :를 쓰면 여러 개의 변수를 선언 가능"  
DATA gv_var3 LIKE gv_var2. "LIKE를 사용하면 이전의 사용한 변수의 Type을 Type으로 참조 가능"
```

Complete Type과 Incomplete Type

- **Complete Type** 은 데이터 길이가 미리 사전 정의된 타입이다.

- **I**, **INT4**, **INT8**, **F**, **D**, **T** 등이 해당된다.

- ★ **D** 와 **T** 의 경우 공백인 데이터를 가져와야 할 때는 조건을 이렇게 비교해야 한다.

```
lv_date EQ ''           "이러면 못 가져온다.  
lv_date EQ '00000000'  "이렇게 해야 가져올 수 있다.  
lv_time EQ ''          "이러면 못 가져온다.  
lv_time EQ '000000'    "이렇게 해야 가져올 수 있다.
```

- **Incomplete Type** 은 데이터 길이를 직접 정의해주어야 하는 타입이다.

- **C** **N** **P** 등이 해당된다.

- **N** 은 Numeric Character로 숫자만 저장할 수 있는 문자열 타입이고, 공백은 0으로 채운다 - ex) `0017`

- **P** 는 부동소수점을 가지는 Packed Number이다.

TYPES

- 데이터 타입과 데이터 길이를 변수로 묶어서 별도로 이름을 지정해 새로운 Type 으로 저장할 수 있다.
 - Complete Type 과 유사한 기능을 한다.

* TYPES로 새로운 TYPE을 선언

```
TYPES gty_perc TYPE p DECIMALS 2.
```

* 새로운 변수에 TYPES로 선언한 TYPE을 지정

```
DATA gv_var TYPE gty_perc.
```

Internal Table

Workarea

- **Internal Table** 은 메모리 내에서 테이블 데이터를 임시로 저장하는 테이블 구조(배열 구조)이다.
 - 데이터베이스 테이블과 유사하게 여러 행(row)의 데이터를 담을 수 있으며, LOOP, READ 등으로 처리한다.
- **Workarea** 는 Internal Table의 단일 행(single row)을 처리하기 위한 구조체이다.
 - Internal Table에서 데이터를 읽거나 쓸 때 임시 작업 공간으로 사용되며, 한 번에 한 행만 담는다.

```
DATA gt_itab TYPE TABLE OF TransparentTable WITH HEADER LINE.  
DATA gt_itab2 LIKE TABLE OF gt_itab.
```

```
DATA: BEGIN OF gs_itab3,  
      "각 필드를 이 부분에서 정의  
      END OF gs_itab3,  
      gt_itab3 LIKE TABLE OF gs_itab3.
```

Internal Table

Workarea

★ Workarea 와 Transparent Table 은 모두 Structure Type 이라고 부른다.

▫ 둘 다 "어떤 필드들로 구성되어 있는가"를 정의하기 때문

★ HEADER LINE 이 있는 Internal Table 은 별도의 Workarea 가 없어도 APPEND 를 다음과 같이 할 수 있다.

```
DATA gt_itab1 TYPE TABLE OF carrid WITH HEADER LINE.
```

```
DATA gt_itab2 TYPE TABLE OF carrid.
```

```
DATA gs_itab2 TYPE carrid.
```

```
APPEND gs_itab2 TO gt_itab2 "원래 방식, 헤더라인이 없으면 별도의 Workarea 필요
```

```
APPEND gt_itab1. "Header Line이 있으면 이렇게 가능
```

```
CLEAR: gt_itab2. "대괄호를 안 붙이면 헤더라인 초기화
```

```
CLEAR: gt_itab2[]. "대괄호를 붙이면 테이블 초기화
```

SELECT-OPTIONS

PARAMETER

- **SELECT-OPTIONS** 는 **Internal Table** 이며, 범위 조건을 입력받는 필드로 4개의 Column(Field)을 가지고 있다.
 - **LOW** : ~부터, **`필수`**로 입력해야 하는 값
 - **HIGH** : ~까지, OPTION에 따라 **`선택`**으로 입력하는 값
 - **SIGN** : I(Include) / E(Exclude) , 포함할 것인지 제외할 것인지 결정
 - **OPTION** : 연산자의 정보 (EQ, GT, LE, BT 등)
- **PARAMETER** 는 단일 값을 입력받는 입력 필드, 사용자로부터 하나의 값만 받을 수 있다.
 - **공백으로 입력할 시 공백인 값을 찾는 것을 조건으로 설정해버리므로, 필수 입력값이다.**

SIGN E OPTION EQ ≠ SIGN I OPTION NE

- 단일 조건이었다면 두 조건은 같은 표현이지만, `SELECT-OPTIONS` 이 여러 조건이 APPEND 될 시 이야기가 다르다.
- SIGN E 는 SIGN I 보다 우선 순위가 높으며, SIGN I 와 같이 조건으로 사용된다면 **AND**로 처리된다.
 - SIGN I 는 SIGN I 만으로 조건들이 구성되면 **OR**로 처리된다.

"테이블에 empno라는 필드가 있고, 1, 2, 3, 4, 5란 데이터가 담겨있다고 가정

LOW = 3 SIGN E OPTION EQ

LOW = 5 SIGN I OPTION EQ

" ⇒ 5인 것을 포함하고(AND), 3은 제외하여 5만 선택됨

LOW = 3 SIGN I OPTION NE

LOW = 5 SIGN I OPTION EQ

" ⇒ 3이 '아닌 것'을 포함하거나 혹은(OR) 5인 것을 포함하므로 1, 2, 4, 5가 선택됨

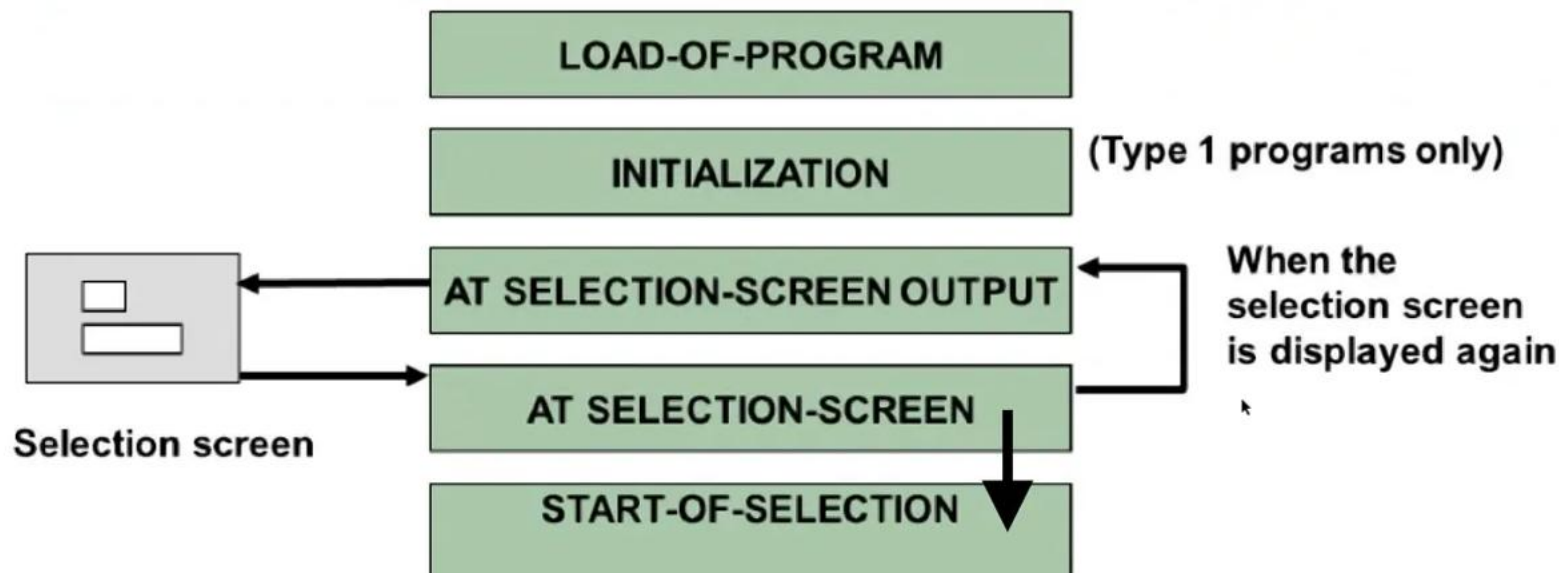
REPORT PROGRAM

- TYPE 1 PROGRAM 이며, Executable Program 이라고도 부른다.
- SELECTION-SCREEN (1000번 스크린)을 가지고 있다. 없을 수도 있지만 거의 예외가 없을 정도로 다 가지고 있다.
- INITIALIZATION AT SELECTION SCREEN START-OF-SELECTION 등의 `Report Event`를 가진다.

BC405

Figure 12: ABAP Events related to Selection Screens

Unit 2 Lesson 1



INITIALIZATION

AT SELECTION SCREEN

START-OF-SELECTION

- **INITIALIZATION** : 각 변수의 초기값을 설정하는 단계
 - SELECTION-SCREEN 필드의 초기값, Workarea 등 모든 변수의 초기값은 이 이벤트에서 할당한다.
- **AT SELECTION SCREEN** : 입력 필드의 입력값들이 유효한지 검증하는 단계
 - SELECTION-SCREEN에 값을 입력하고 실행 버튼을 눌렀을 때 실행되는 이벤트. 즉 **PAI (Process After Input)**이다.
- **AT SELECTION SCREEN OUTPUT** : SELECTION-SCREEN이 출력되기 전 출력 속성을 컨트롤한다.
 - PARAMETER, SELECT-OPTIONS 등의 입력 필드에 읽기 전용, 숨김 처리 등을 한다.
 - **SELECTION-SCREEN이 실행되기 전에** 실행되는 이벤트, 즉 **PBO (Process Before Output)**이다.
- **START-OF-SELECTION** : 실제 프로그램의 시작을 알리는 단계, **필수로 입력해야 하는 이벤트**이다.

MODIFY SCREEN 실습

- SELECTION-SCREEN에는 파라미터 3개와 2개의 체크 박스를 설치합니다.

- pa_carr(sbook-carrid), pa_conn(spfli-connid), pa_fldt(sflight-fldate)
- pa_edit (Read Only), pa_actv (Active / Hide)

- Read Only에 체크하면 Airline과 Connection Number이 읽기 전용으로 바뀌고, Show / Hide에 체크하면 Booking Number이 보여지거나 숨김 처리되게끔 해주세요.
- INCLUDE(모듈화)와 PERFORM(서브루틴)을 사용하여 코드를 작성합니다.

Condition

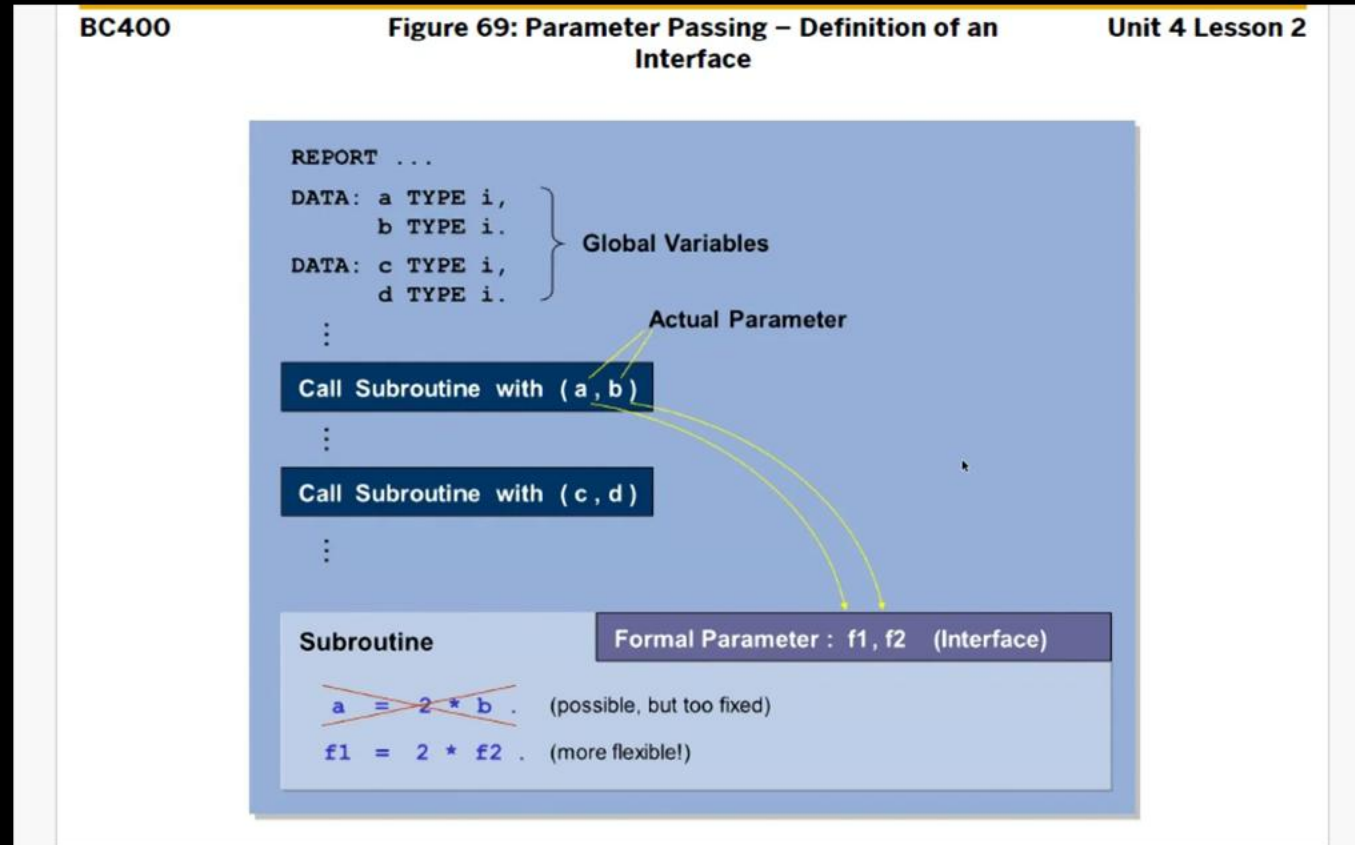
Airline	LH
Connection Number	401
Booking number	

to

☒ Read Only
☐ Show / Hide

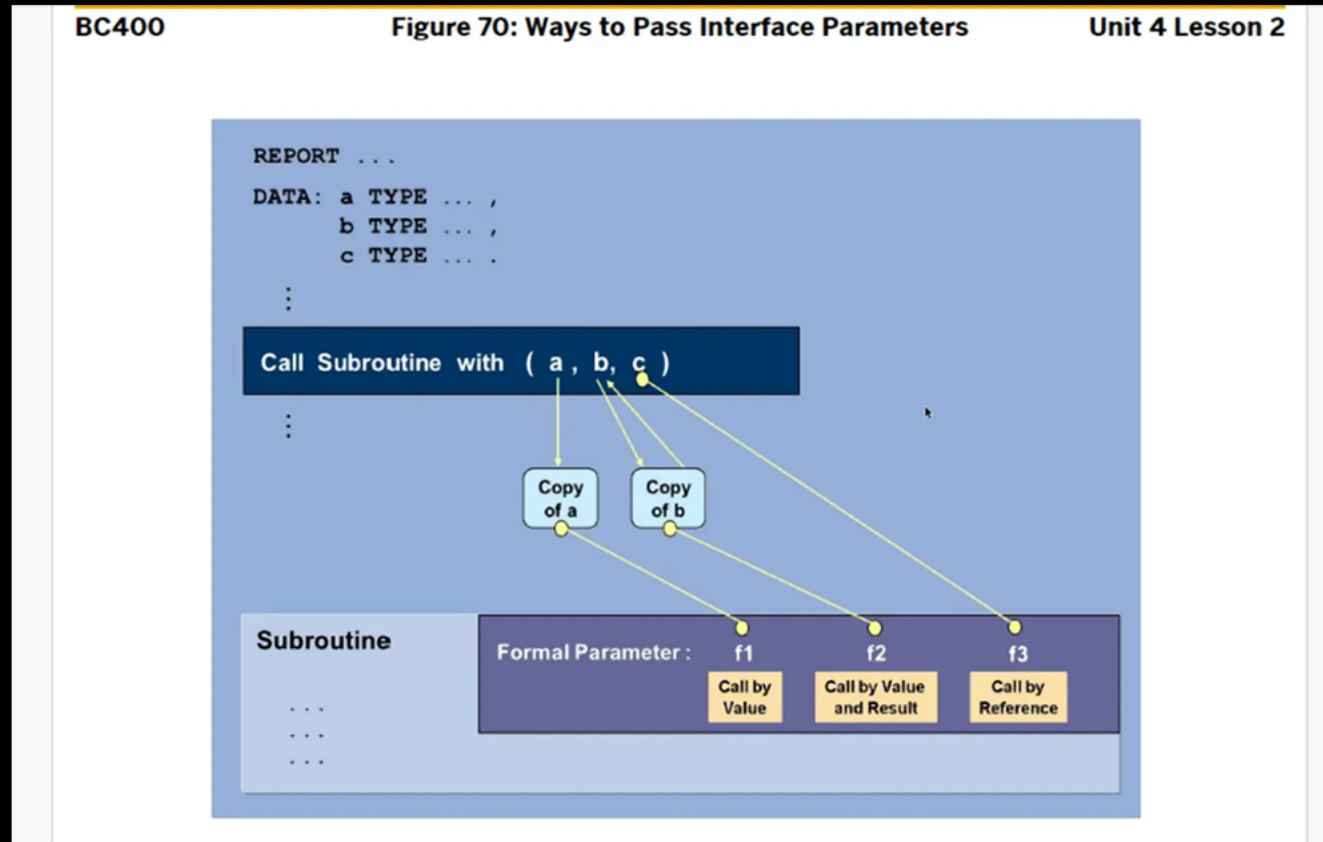
Subroutine With Parameter

- 서브루틴에 매개변수를 넣어 활용할 수 있다.
- 서브루틴에 전역변수를 사용하는 것보다 매개변수를 사용하는 것이 훨씬 유연하고 유지보수에 좋은 코딩이다.



Call By Value / Value & Result / Reference

- 서브루틴을 호출하여 실 매개변수를 넣었을 때 서브루틴 호출 이후 실 매개변수의 변화여부에 따라 나뉜다.
- 값에 의한 호출은 실 매개변수가 변하지 않고, 나머지 CBVR과 CBR은 실 매개변수의 값이 변한다.



요약, 그리고 이것들은 언제 사용하는가?

- Call By Value는 고객정보 조회, 출력 등 동작을 수행하고 나서 입력값의 값이 변하지 않아야 할 때 쓰인다.
- Call By Reference는 **대용량 데이터**를 다뤄야 할 때 자주 사용한다.
 - 10만 건 이상의 데이터를 집계할 때 Call By Value를 쓰면 데이터를 10만 건 이상을 복사해야하기 때문
 - 그래서 속도와 메모리를 중시하는 실무에서 가장 많이 사용하는 방식이기도 하다

" 10만 건 데이터 생성 (예시)

```
DO 100000 TIMES.  
    gt_sales-sales_id = sy-index.  
    gt_sales-amount = sy-index * 1000.  
    gt_sales-region = 'SEOUL'.  
    APPEND gt_sales.  
ENDDO.
```

" ❌ 나쁜 예: CBV 사용 (10만 건을 복사!)
" PERFORM calculate_total_sales_slow
" USING VALUE(gt_sales) " 메모리 낭비!
" CHANGING gv_total.

" ✅ 좋은 예: CBR 사용 (주소만 전달)
DATA: gv_total TYPE p DECIMALS 2.
PERFORM calculate_total_sales_fast
 USING gt_sales " 복사 없이 주소만 전달
 CHANGING gv_total.

구분	ABAP 키워드	원본 변경	메모리	성능
CBV	VALUE(param)	X 안됨	복사본 생성	느림
CBV&R	CHANGING	✓ 됨 (종료 시)	복사본 생성	중간
CBR	USING param	✓ 됨 (실시간)	주소 공유	빠름

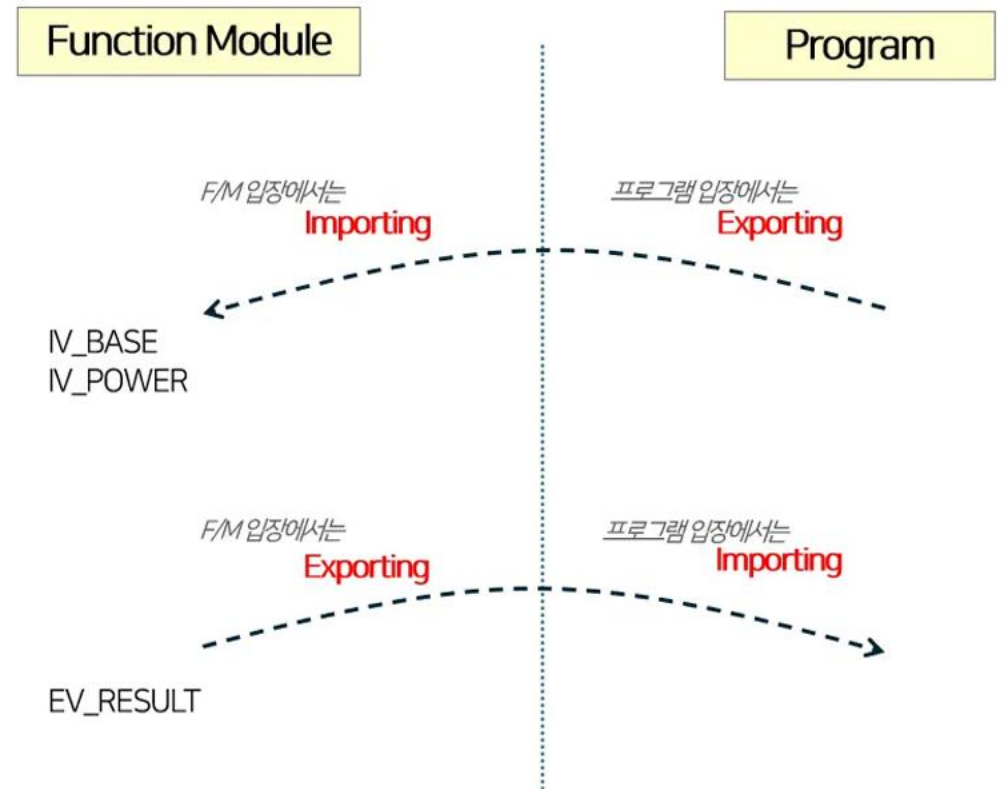
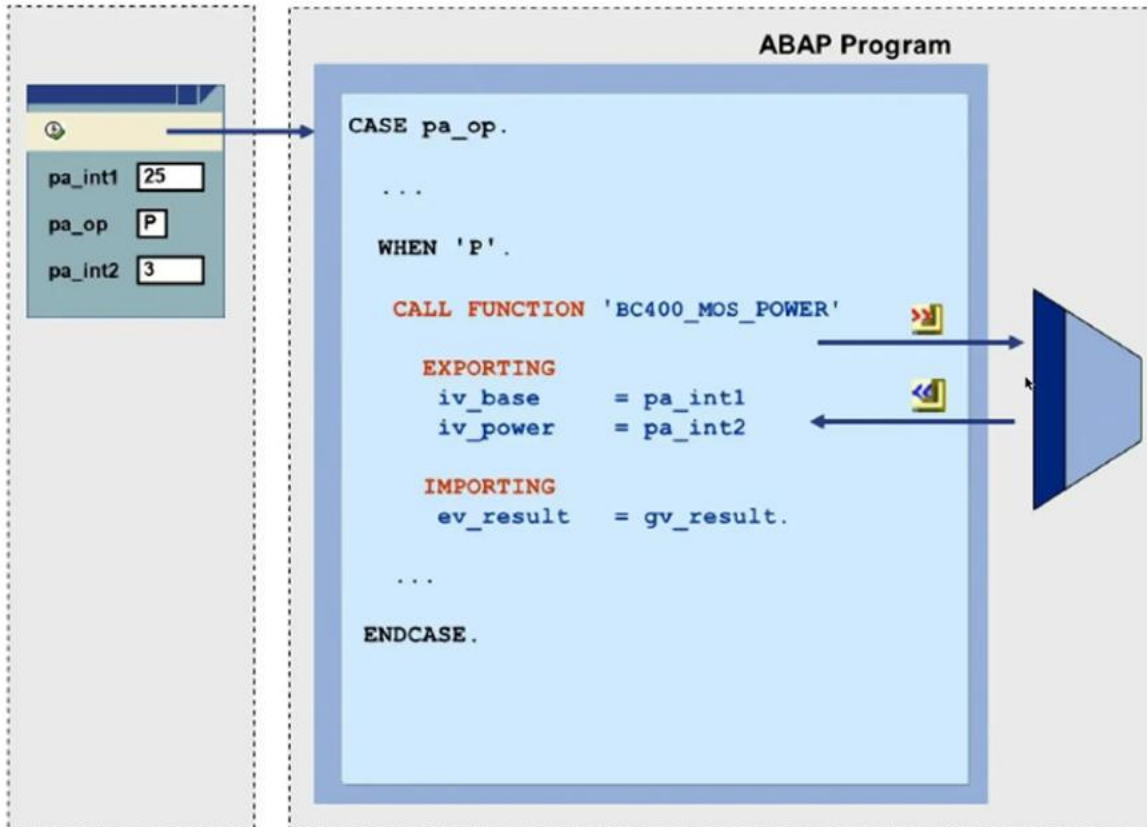
Function Group VS ABAP Report Program

- **Function Group** : **Function Module**을 통해 다른 프로그램에서 `CALL FUNCTION` 으로 호출 가능
 - 나중에 배울 **Module Pool(Dialog Programming)**의 기반으로 사용
 - 여러 프로그램에서 재사용 가능 → **Reusable(재사용 가능), Shared components (공유 컴포넌트)**
- **ABAP Report Program** : **기본적으로 독립 실행용 (SE38에서 F8)**
 - 다른 프로그램에서 **SUBMIT** 명령으로 호출은 가능하지만, 일반적인 재사용 패턴은 아님
 - 주로 단독으로 실행되는 용도 → **Standalone(단독적), Executable programs**
 - 오해하지 말아야 할 것 : 다른 Type의 Program(Function Group, Include Program 등)들은 재사용이 가능한 프로그램들도 있음
Module Pool, Class Pool, Subroutine Pool

모듈화 대상	Program	Function Group
타입	Local Type	Global Type
기능 단위	PERFORM (Subroutine)	Function Module
전역 변수	TOP Include	TOP Include
루틴 정의	FORM Routine (F01)	FORM Routine (F01)

Function Module

- 아래 사진을 기준으로 내보내는 데이터는 EXPORTING, 받는 데이터는 IMPORTING에 적어둔다.
- 유일하게 Tables 파라미터는 양방향으로 데이터를 주고받는다. (받을 수도, 줄 수도, 수정도 가능)
 - 즉, Tables 파라미터는 Call By Reference로 작동한다. (Call By Value 사용불가)



Function Module

- SQL Query 문은 Function Module 내부에 있지만, **실제 테이블의 저장공간은 호출자인 Program**에 있음.
- 즉 TABLES는 **Program의 Internal Table**을 참조(Reference)하는 포인터(주소를 가르키는 역할)



Function Module 실습문제

- 어제 오후에 했던 Function Module 실습에서 난이도를 낮추어서 만들었습니다. (난이도 낮추었으니 복붙은 X)
- SELECT-SCREEN에 **Airline**을 입력받는 **파라미터**를 하나 만듭니다. (**spfli-carrid**, 기본값 'AA')
- **파라미터의 입력값을 WHERE 조건으로 받아 SPFLI 테이블을 조회하는 SQL**를 수행하는 **Function Module**을 만들고, **서브쿼리를 PERFORM으로 선언하여 서브쿼리 내에서 Function Module을 호출**하세요.
- **파라미터의 초기값은 ABAP Report Event를 사용해 초기화**하고, **SELECT-SCREEN** 및 **전역 변수** 등은 꼭 **INCLUDE PROGRAM**을 사용하여 코딩합니다.
- 성공 시 몇 건을 조회 성공했는지, 실패 시에는 데이터를 찾지 못했다는 오류 메시지를 출력하세요.

Condition

Airline

AA



Output

Operand	MANDT	CARRID	CONNID	COUNTRYFR	CITYFROM	AIRPFROM	COUNTRYTO	CITYTO	AIRPTO	FLTIME	DEPTIME	ARRTIME	DISTANCE	DISTID	F
100	AA	0017	US	NEW YORK	JFK	US	SAN FRANCISCO	SFO	361	11:00:00	14:01:00	2572.0	MI		
100	AA	0064	US	SAN FRANCISCO	SFO	US	NEW YORK	JFK	321	09:00:00	17:21:00	2572.0	MI		